

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Chitra Shree K S (1WA24CS100)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Chitra Shree K S (**1WA24CS100**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Shwetha K S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-10
2	Implement Johnson Trotter algorithm to generate permutations.	11-13
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	14-17
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	18-21
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	22-24
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	25-28
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	29-32
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	33-36
9	Implement Fractional Knapsack using Greedy technique.	37-39
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	40-42
11	Implement "N-Queens Problem" using Backtracking.	43-45

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 100

int V[MAX][MAX];
int indegree[MAX];
int visited[MAX];
int TP[MAX];

int main() {
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &V[i][j]);
        }
    }
    for (int i = 0; i < n; i++) {
        indegree[i] = 0;
        for (int j = 0; j < n; j++) {
            if (V[j][i] == 1) {
                indegree[i]++;
            }
        }
    }
```

```
}
```

```
for (int i = 0; i < n; i++) {  
    visited[i] = 0;  
}
```

```
int tp_count = 0;
```

```
while (1) {  
    int w = -1;
```

```
    for (int i = 0; i < n; i++) {  
        if (visited[i] == 0 && indegree[i] == 0) {  
            w = i;  
            break;  
        }  
    }
```

```
    if (w == -1)  
        break;
```

```
    TP[tp_count++] = w;  
    visited[w] = 1;
```

```
    for (int i = 0; i < n; i++) {  
        if (V[w][i] == 1) {  
            indegree[i]--;  
        }  
    }
```

```

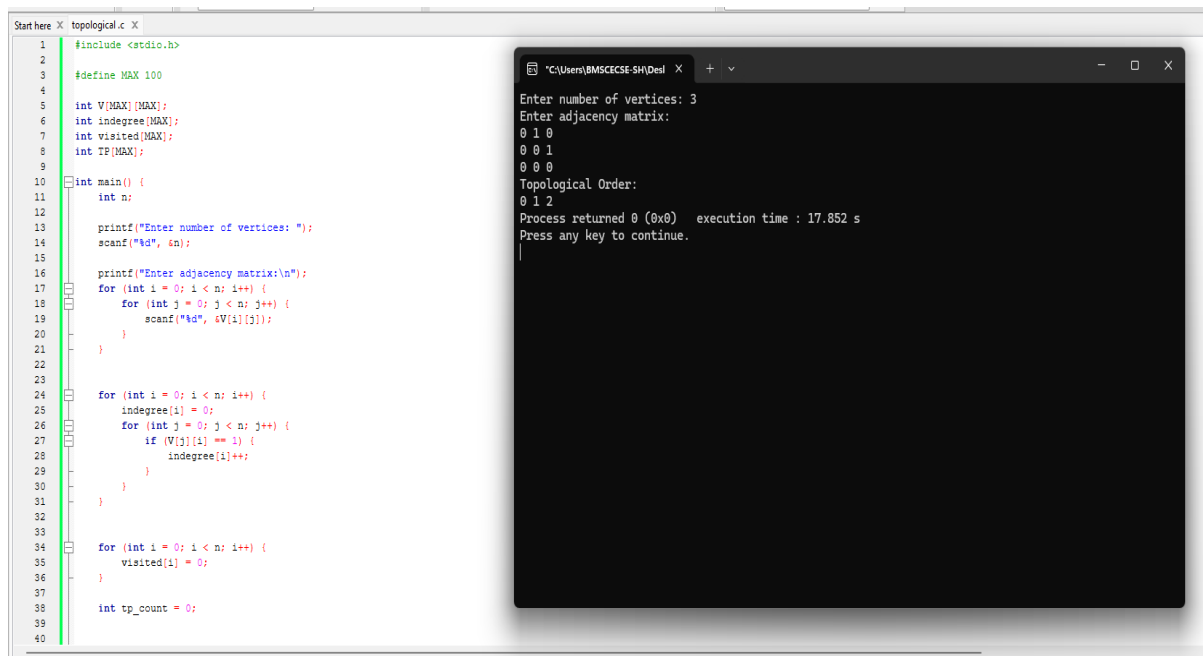
    }
}

if (tp_count < n) {
    printf("No Topological Sequence (Cycle detected)\n");
} else {
    printf("Topological Order:\n");
    for (int i = 0; i < tp_count; i++) {
        printf("%d ", TP[i]);
    }
}

return 0;
}

```

Output:



The image shows a C++ IDE with a source code editor on the left and a terminal window on the right. The source code implements a topological sort algorithm using Kahn's algorithm. It defines a maximum number of vertices (MAX) as 100. The main function prompts the user to enter the number of vertices (n) and the adjacency matrix. It then calculates the in-degree for each vertex and processes the vertices with zero in-degree to produce the topological order. The terminal window shows the execution of the program with the input n=3 and an adjacency matrix [[0,1,0],[0,0,1],[0,0,0]]. The output is the topological order 0 1 2. The process returned 0 (0x0) and the execution time was 17.852 s.

```

1  #include <stdio.h>
2
3  #define MAX 100
4
5  int V[MAX][MAX];
6  int indegree[MAX];
7  int visited[MAX];
8  int TP[MAX];
9
10 int main() {
11     int n;
12
13     printf("Enter number of vertices: ");
14     scanf("%d", &n);
15
16     printf("Enter adjacency matrix:\n");
17     for (int i = 0; i < n; i++) {
18         for (int j = 0; j < n; j++) {
19             scanf("%d", &V[i][j]);
20         }
21     }
22
23
24     for (int i = 0; i < n; i++) {
25         indegree[i] = 0;
26         for (int j = 0; j < n; j++) {
27             if (V[j][i] == 1) {
28                 indegree[i]++;
29             }
30         }
31     }
32
33
34     for (int i = 0; i < n; i++) {
35         visited[i] = 0;
36     }
37
38     int tp_count = 0;
39
40

```

```

Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
0 0 0
Topological Order:
0 1 2
Process returned 0 (0x0)   execution time : 17.852 s
Press any key to continue.

```

Leet Code exercises on topological sorting

1. 207

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {

    int indegree[2000] = {0};
    int graph[2000][2000] = {0};

    // Build graph + indegree
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];

        if (graph[b][a] == 0) {
            graph[b][a] = 1;
            indegree[a]++;
        }
    }

    // Queue for BFS
    int queue[2000];
    int front = 0, rear = 0;

    // Push all nodes with indegree 0
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    int count = 0;

    // BFS (Kahn's Algorithm)
    while (front < rear) {
        int course = queue[front++];
        count++;

        for (int i = 0; i < numCourses; i++) {
            if (graph[course][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0) {
                    queue[rear++] = i;
                }
            }
        }
    }
}
```

```

        return (count == numCourses);
    }
    int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize, int* returnSize) {

        int indegree[2000] = {0};
        int graph[2000][2000] = {0};

        // Build graph + indegree
        for (int i = 0; i < prerequisitesSize; i++) {
            int a = prerequisites[i][0];
            int b = prerequisites[i][1];

            if (graph[b][a] == 0) {
                graph[b][a] = 1;
                indegree[a]++;
            }
        }

        // Queue for BFS
        int queue[2000];
        int front = 0, rear = 0;

        // Result array
        int* order = (int*)malloc(sizeof(int) * numCourses);
        int idx = 0;

        // Push all indegree 0 nodes
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) {
                queue[rear++] = i;
            }
        }

        // BFS (Kahn's Algorithm)
        while (front < rear) {
            int course = queue[front++];
            order[idx++] = course;

            for (int i = 0; i < numCourses; i++) {
                if (graph[course][i] == 1) {
                    indegree[i]--;
                    if (indegree[i] == 0) {
                        queue[rear++] = i;
                    }
                }
            }
        }
    }
}

```



```

// If not all courses are processed → cycle exists
if (idx != numCourses) {
    *returnSize = 0;
    return NULL;
}

*returnSize = numCourses;
return order;
}

```

Output:

207. Course Schedule Solved

Medium Topics Companies Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:
Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:
Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are unique.

Testcase Test Result Runtime: 6 ms

Accepted

Case 1 **Case 2**

Input

`numCourses =`
`2`

`prerequisites =`
`[[1,0]]`

Output
`true`

Expected
`true`

Contribute a testcase

2. 210

```

int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize,
int* prerequisitesColSize, int* returnSize) {

    int indegree[2000] = {0};
    int graph[2000][2000] = {0};
    int queue[2000], front = 0, rear = 0;

    int* order = (int*)malloc(sizeof(int) * numCourses);
    int idx = 0;

    // build graph + indegree
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0], b = prerequisites[i][1];
        if (!graph[b][a]) {
            graph[b][a] = 1;
            indegree[a]++;
        }
    }
}

```

```

// push 0 indegree nodes
for (int i = 0; i < numCourses; i++)
    if (indegree[i] == 0)
        queue[rear++] = i;

// BFS
while (front < rear) {
    int u = queue[front++];
    order[idx++] = u;

    for (int v = 0; v < numCourses; v++) {
        if (graph[u][v] && --indegree[v] == 0)
            queue[rear++] = v;
    }
}

if (idx != numCourses) {
    *returnSize = 0;
    return NULL;
}

*returnSize = numCourses;
return order;
}

```

Output:

210. Course Schedule II

Medium Topics Companies Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return an **empty array**.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `[0,1]`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.

Example 2:

Input: `numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`
Output: `[0,2,1,3]`
Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0. So one correct course order is `[0,1,2,3]`. Another correct ordering is `[0,2,1,3]`.

Example 3:

Input: `numCourses = 1, prerequisites = []`
Output: `[0]`

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`

Solved

```

1 int* findOrder(int numCourses, int** prerequisites, int prerequisitesize, int* prerequisitesColSize, int* returnSize) {
2
3     int indegree[2000] = {0};
4     int graph[2000][2000] = {0};
5     int queue[2000], front = 0, rear = 0;
6
7     int* order = (int*)malloc(sizeof(int) * numCourses);
8     int idx = 0;
9
10    // build graph + indegree

```

Testcase Test Result

Accepted Runtime: 5 ms

Case 1 Case 2 Case 3

Input

`numCourses =`
`2`

`prerequisites =`
`[[1,0]]`

Output

`[0,1]`

Expected

`[0,1]`

Contribute a testcase

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", perm[i]);
    printf("\n");
}

int getMobile(int perm[], int dir[], int n) {
    int mobile = 0;
    int mobile_index = -1;

    for (int i = 0; i < n; i++) {

        if (dir[perm[i] - 1] == LEFT && i != 0) {
            if (perm[i] > perm[i - 1] && perm[i] > mobile) {
                mobile = perm[i];
                mobile_index = i;
            }
        }

        if (dir[perm[i] - 1] == RIGHT && i != n - 1) {
            if (perm[i] > perm[i + 1] && perm[i] > mobile) {
                mobile = perm[i];
                mobile_index = i;
            }
        }
    }
}
```

```

    }

    return mobile_index;
}

void johnsonTrotter(int n) {
    int perm[n];
    int dir[n];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(perm, n);
    while (1) {
        int mobile_index = getMobile(perm, dir, n);
        if (mobile_index == -1)
            break;
        int mobile = perm[mobile_index];
        if (dir[mobile - 1] == LEFT) {
            int temp = perm[mobile_index];
            perm[mobile_index] = perm[mobile_index - 1];
            perm[mobile_index - 1] = temp;
            mobile_index--;
        } else {
            int temp = perm[mobile_index];
            perm[mobile_index] = perm[mobile_index + 1];
            perm[mobile_index + 1] = temp;
            mobile_index++;
        }
        for (int i = 0; i < n; i++) {
            if (perm[i] > mobile) {

```

```

        dir[perm[i] - 1] *= -1;
    }
}

printPermutation(perm, n);
}
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    johnsonTrotter(n);

    return 0;
}

```

Output:

```

1  #include <stdio.h>
2
3  #define LEFT -1
4  #define RIGHT 1
5
6  void printPermutation(int perm[], int n) {
7      for (int i = 0; i < n; i++)
8          printf("%d ", perm[i]);
9      printf("\n");
10 }
11
12 int getMobile(int perm[], int dir[], int n) {
13     int mobile = 0;
14     int mobile_index = -1;
15
16     for (int i = 0; i < n; i++) {
17         if (dir[perm[i] - 1] == LEFT && i != 0) {
18             if (perm[i] > perm[i - 1] && perm[i] > mobile) {
19                 mobile = perm[i];
20                 mobile_index = i;
21             }
22         }
23         if (dir[perm[i] - 1] == RIGHT && i != n - 1) {
24             if (perm[i] > perm[i + 1] && perm[i] > mobile) {
25                 mobile = perm[i];
26                 mobile_index = i;
27             }
28         }
29     }
30
31     return mobile_index;
32 }
33
34 void johnsonTrotter(int n) {
35     int perm[n];
36     int dir[n];
37
38     for (int i = 0; i < n; i++) {
39         perm[i] = i + 1;
40         dir[i] = LEFT;
41     }
42
43     printPermutation(perm, n);
44
45     while (1) {
46         int mobile_index = getMobile(perm, dir, n);
47
48         if (mobile_index == -1) break;
49
50         // Swap perm[mobile_index] with perm[mobile_index - 1]
51         int temp = perm[mobile_index];
52         perm[mobile_index] = perm[mobile_index - 1];
53         perm[mobile_index - 1] = temp;
54
55         // Reverse direction of mobile element
56         dir[mobile_index - 1] = -dir[mobile_index - 1];
57
58         printPermutation(perm, n);
59     }
60 }

```

```

C:\Users\BMSCECSE-SH\Desktop > gcc johnson.c -o johnson.exe
C:\Users\BMSCECSE-SH\Desktop > johnson.exe
Enter number of elements: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
Process returned 0 (0x0)   execution time : 1.285 s
Press any key to continue.

```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define MAX 50000
int arr[MAX],temp[MAX];
void merge (int low,int mid,int high)
{
    int i =low;
    int j = mid+1;
    int k = low;

    while(i<=mid&& j<=high )
    {
        if (arr[i]<arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];
```

```

for (i = low; i <= high; i++)
arr[i] = temp[i];
}

void mergesort(int low, int high)
{
if (low < high)
{
int mid = (low + high) / 2;
mergesort(low, mid);
mergesort(mid + 1, high);
merge(low, mid, high);
}
}

int main()
{
int n, i;
clock_t start, end;
double time_taken;

srand(time(NULL));
printf("N\tTime Taken(seconds)\n");

for (n = 10000; n <= 50000; n += 10000)
{

for (i = 0; i < n; i++)
{
arr[i] = rand() % 10000;

```

```

}

start = clock();

mergesort(0, n - 1);

end = clock();

time_taken = (double)(end - start) / CLOCKS_PER_SEC;

printf("%d\t%.10f\n",n,time_taken);

}

return 0;

}

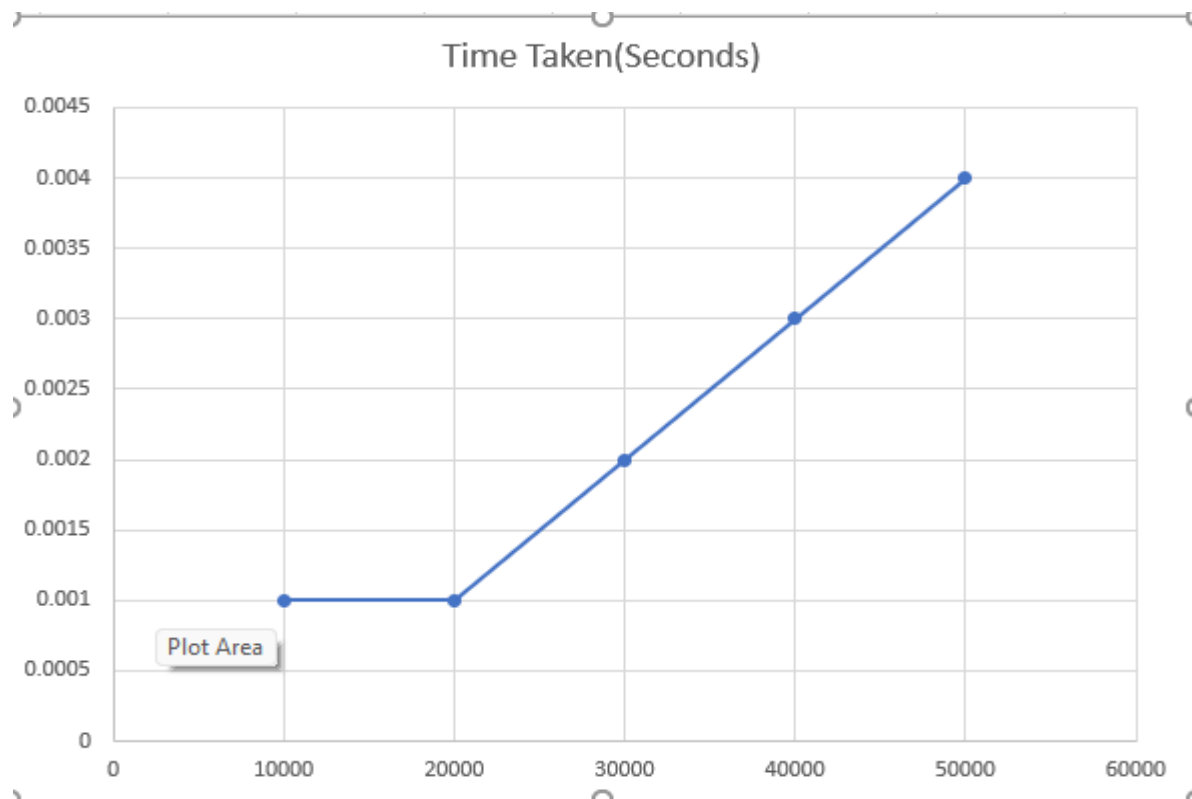
```

Output:

The screenshot shows a C++ IDE with a file named `MERGESORT.C` open. The code is a C++ implementation of the mergesort algorithm. It includes headers for `stdio.h`, `stdlib.h`, and `time.h`. It defines a constant `MAX` as 50000. The `merge` function is a recursive function that sorts an array `arr` between indices `low` and `high`. It uses a temporary array `temp` to store the merged elements. The `mergesort` function is a recursive function that calls `merge` to sort the array. The `main` function takes an integer `n` as input, calls `mergesort` to sort the array, and prints the execution time for each value of `n` from 10000 to 50000. The output window shows the results of the program, which are as follows:

N	Time Taken(seconds)
10000	0.0010000000
20000	0.0010000000
30000	0.0020000000
40000	0.0030000000
50000	0.0040000000

Process returned 0 (0x0) execution time : 0.024 s
Press any key to continue.



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 50000
int arr[MAX];
int partition(int low, int high)
{
    int pivot = arr[low];
    int i = low + 1;
    int j = high;
    int temp;
    while (1)
    {
        while (i <= high && arr[i] <= pivot)
            i++;
        while (arr[j] > pivot)
            j--;
        if (i < j)
        {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
        else
        {
            temp = arr[low];
            arr[low] = arr[j];
            arr[j] = temp;
        }
    }
}
```

```

arr[j] = temp;
return j;
}
}
}

void quicksort(int low, int high)
{
if (low < high)
{
int pos = partition(low, high);
quicksort(low, pos - 1);
quicksort(pos + 1, high);
}
}

int main()
{
int n, i;
clock_t start, end;
double time_taken;

srand(time(NULL));

printf("N\tTime Taken (seconds)\n");

for (n = 10000; n <= 50000; n += 10000)
{
for (i = 0; i < n; i++)
{

```

```

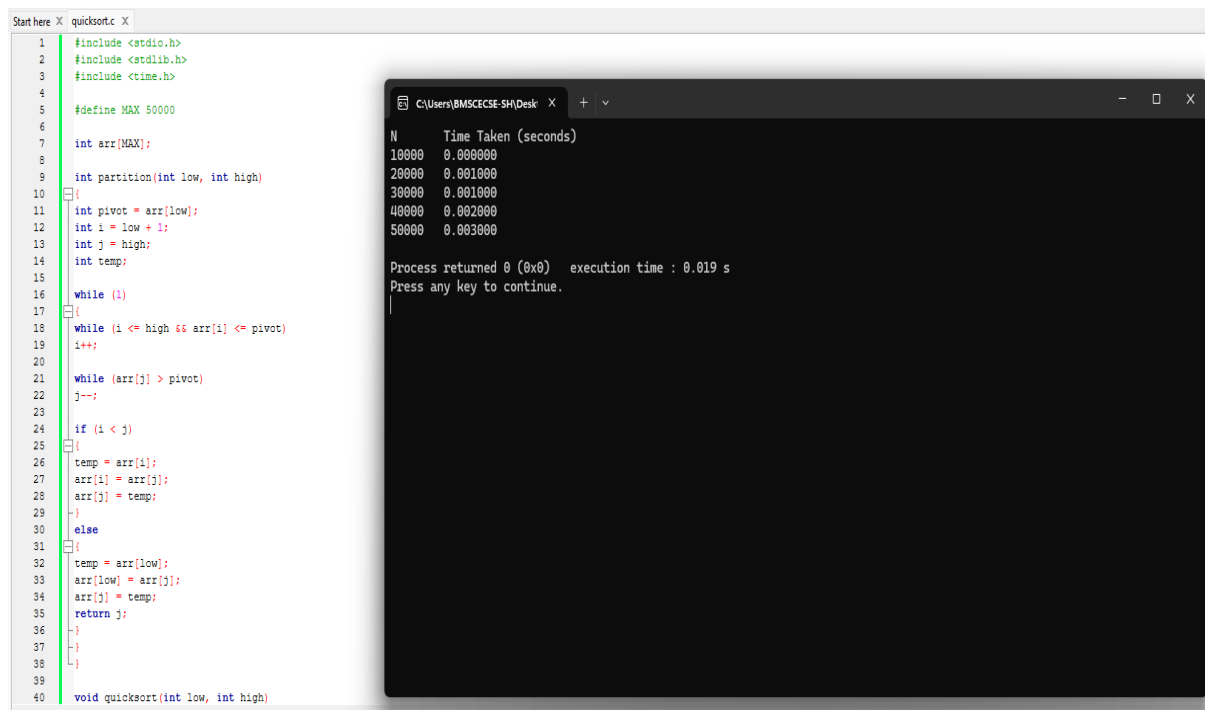
arr[i] = rand() % 10000;
}

start = clock();
quicksort(0, n - 1);
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("%d\t%f\n", n, time_taken);
}

return 0;
}

```

Output:



The screenshot shows a C++ IDE with a file named `quicksort.c` open. The code implements a quicksort algorithm. To the right, a terminal window displays the output of the program, showing the time taken for different values of `N` (10000, 20000, 30000, 40000, 50000). The output also shows the process returned 0 (0x0) and the execution time was 0.019 s.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define MAX 50000
6
7  int arr[MAX];
8
9  int partition(int low, int high)
10 {
11     int pivot = arr[low];
12     int i = low + 1;
13     int j = high;
14     int temp;
15
16     while (1)
17     {
18         while (i <= high && arr[i] <= pivot)
19             i++;
20
21         while (arr[j] > pivot)
22             j--;
23
24         if (i < j)
25         {
26             temp = arr[i];
27             arr[i] = arr[j];
28             arr[j] = temp;
29         }
30         else
31         {
32             temp = arr[low];
33             arr[low] = arr[j];
34             arr[j] = temp;
35             return j;
36         }
37     }
38 }
39
40 void quicksort(int low, int high)

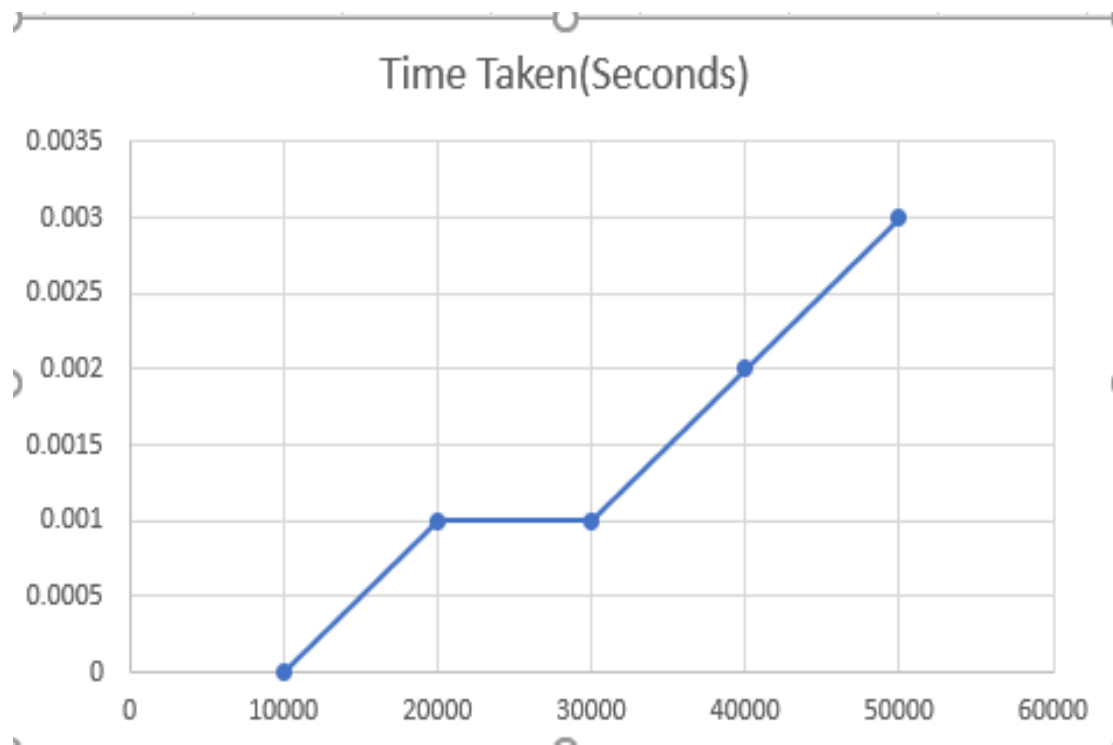
```

```

N      Time Taken (seconds)
10000  0.000000
20000  0.001000
30000  0.001000
40000  0.002000
50000  0.003000

Process returned 0 (0x0)   execution time : 0.019 s
Press any key to continue.

```



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void heapify(int arr[], int n, int i) {
```

```
    int largest = i;
```

```
    int left = 2*i + 1;
```

```
    int right = 2*i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;
```

```
    if (largest != i) {
```

```
        int temp = arr[i];
```

```
        arr[i] = arr[largest];
```

```
        arr[largest] = temp;
```

```
        heapify(arr, n, largest);
```

```
    }
```

```
}
```

```

void heapSort(int arr[], int n) {

    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n-1; i > 0; i--) {

        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter elements:\n");

```

```

for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

clock_t start, end;

start = clock();
heapSort(arr, n);
end = clock();

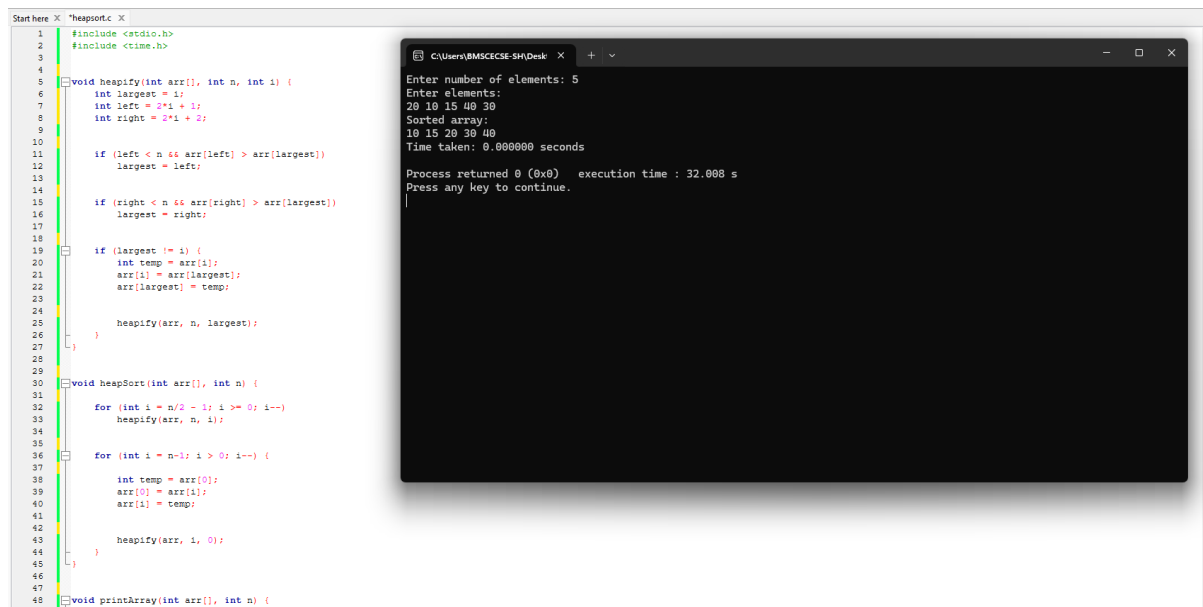
printf("Sorted array:\n");
printArray(arr, n);

double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Time taken: %f seconds\n", time_taken);

return 0;
}

```

Output:



The screenshot shows a C++ IDE with two windows. The left window displays the source code for 'heapSort.c', which includes headers, a heapify function, a heapSort function, and a printArray function. The right window is a terminal output showing the program's execution. It prompts for the number of elements (5) and the elements themselves (20 18 15 40 30). It then displays the sorted array (18 15 20 30 40) and the time taken (0.000000 seconds). The process returned 0 (0x0) and the execution time was 32.008 s. The user is prompted to press any key to continue.

```

Start here X "heapSort.c" X
1 #include <stdio.h>
2 #include <time.h>
3
4
5 void heapify(int arr[], int n, int i) {
6     int largest = i;
7     int left = 2*i + 1;
8     int right = 2*i + 2;
9
10
11     if (left < n && arr[left] > arr[largest])
12         largest = left;
13
14     if (right < n && arr[right] > arr[largest])
15         largest = right;
16
17     if (largest != i) {
18         int temp = arr[i];
19         arr[i] = arr[largest];
20         arr[largest] = temp;
21
22         heapify(arr, n, largest);
23     }
24 }
25
26 void heapSort(int arr[], int n) {
27     for (int i = n/2 - 1; i >= 0; i--)
28         heapify(arr, n, i);
29
30     for (int i = n-1; i > 0; i--) {
31         int temp = arr[0];
32         arr[0] = arr[i];
33         arr[i] = temp;
34
35         heapify(arr, i, 0);
36     }
37 }
38
39 void printArray(int arr[], int n) {
40     for (int i = 0; i < n; i++)
41         printf("%d ", arr[i]);
42     printf("\n");
43 }
44
45 int main() {
46     int n;
47     printf("Enter number of elements: ");
48     scanf("%d", &n);
49     int arr[n];
50     printf("Enter elements:\n");
51     for (int i = 0; i < n; i++)
52         scanf("%d", &arr[i]);
53     heapSort(arr, n);
54     printf("Sorted array:\n");
55     printArray(arr, n);
56     double time_taken = ((double)(clock() - start)) / CLOCKS_PER_SEC;
57     printf("Time taken: %f seconds\n", time_taken);
58     return 0;
59 }

```

```

C:\Users\BMSCECSE-SH\Desktop X + -
Enter number of elements: 5
Enter elements:
20 18 15 40 30
Sorted array:
18 15 20 30 40
Time taken: 0.000000 seconds
Process returned 0 (0x0)   execution time : 32.008 s
Press any key to continue.

```


Lab program 6:**Implement 0/1 Knapsack problem using dynamic programming.**`#include <stdio.h>``int max(int a, int b) {``return (a > b) ? a : b;``}``int knapsack(int W, int wt[], int val[], int n) {``int dp[n+1][W+1];``for (int i = 0; i <= n; i++) {``for (int w = 0; w <= W; w++)``if (i == 0 || w == 0)``dp[i][w] = 0;``else if (wt[i-1] <= w)``dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]],``dp[i-1][w]);`

```
        else

            dp[i][w] = dp[i-1][w];

        }

    }

    return dp[n][W];

}

int main() {

    int n, W;

    printf("Enter number of items: ");

    scanf("%d", &n);

    int val[n], wt[n];

    printf("Enter values of items:\n");

    for (int i = 0; i < n; i++)

        scanf("%d", &val[i]);
```

```
printf("Enter weights of items:\n");
```

```
for (int i = 0; i < n; i++)
```

```
scanf("%d", &wt[i]);
```

```
printf("Enter capacity of knapsack: ");
```

```
scanf("%d", &W);
```

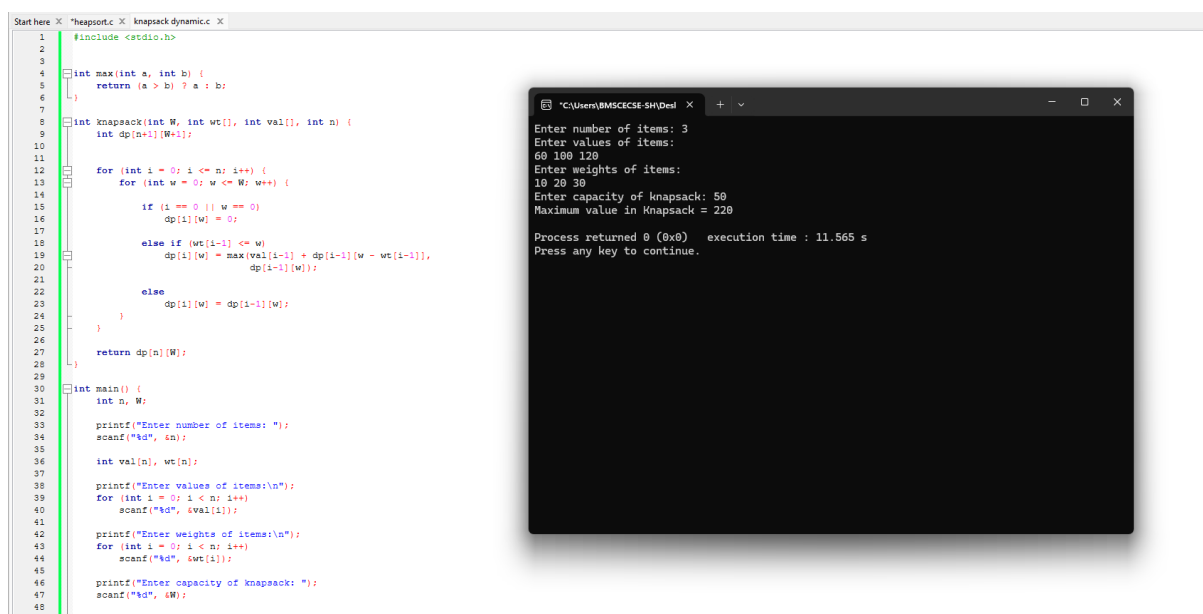
```
int result = knapsack(W, wt, val, n);
```

```
printf("Maximum value in Knapsack = %d\n", result);
```

```
return 0;
```

```
}
```

Output:



The screenshot displays a C++ IDE with two windows. The left window, titled 'knapsack dynamic.c', shows the source code for a knapsack problem. The code includes a helper function 'max' to find the maximum of two integers, a recursive function 'knapsack' that uses a 2D array 'dp' for memoization, and a 'main' function that prompts the user for the number of items, their values, weights, and the knapsack capacity. The right window shows the program's execution output, where the user has entered 3 items with values [60, 100, 120] and weights [10, 20, 30], and a knapsack capacity of 50. The program outputs the maximum value as 220 and reports an execution time of 11.565 seconds.

```
1 #include <stdio.h>
2
3
4 int max(int a, int b) {
5     return (a > b) ? a : b;
6 }
7
8 int knapsack(int W, int wt[], int val[], int n) {
9     int dp[n+1][W+1];
10
11     for (int i = 0; i <= n; i++) {
12         for (int w = 0; w <= W; w++) {
13             if (i == 0 || w == 0)
14                 dp[i][w] = 0;
15             else if (wt[i-1] <= w)
16                 dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]],
17                               dp[i-1][w]);
18             else
19                 dp[i][w] = dp[i-1][w];
20         }
21     }
22     return dp[n][W];
23 }
24
25 int main() {
26     int n, W;
27
28     printf("Enter number of items: ");
29     scanf("%d", &n);
30
31     int val[n], wt[n];
32
33     printf("Enter values of items:\n");
34     for (int i = 0; i < n; i++)
35         scanf("%d", &val[i]);
36
37     printf("Enter weights of items:\n");
38     for (int i = 0; i < n; i++)
39         scanf("%d", &wt[i]);
40
41     printf("Enter capacity of knapsack: ");
42     scanf("%d", &W);
43
44     int result = knapsack(W, wt, val, n);
45 }
```

```
Enter number of items: 3
Enter values of items:
60 100 120
Enter weights of items:
10 20 30
Enter capacity of knapsack: 50
Maximum value in Knapsack = 220

Process returned 0 (0x0)   execution time : 11.565 s
Press any key to continue.
```

Leetcode on 0/1 Knapsack

```
#include <stdlib.h>

int findTargetSumWays(int* nums, int numsSize, int target) {

    int sum = 0;

    for (int i = 0; i < numsSize; i++)
        sum += nums[i];

    if ((target + sum) % 2 != 0 || abs(target) > sum)
        return 0;

    int subsetSum = (target + sum) / 2;

    int dp[subsetSum + 1];

    for (int i = 0; i <= subsetSum; i++)
        dp[i] = 0;

    dp[0] = 1;

    for (int i = 0; i < numsSize; i++) {
        for (int j = subsetSum; j >= nums[i]; j--) {
            dp[j] += dp[j - nums[i]];
        }
    }

    return dp[subsetSum];
}
```

Output:

494. Target Sum Solved

Medium Topics Comments

You are given an integer array `nums` and an integer `target`.

You want to build an **expression** out of `nums` by adding one of the symbols `+` and `-` before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 3]`, you can add a `+` before `2` and a `-` before `3` and concatenate them to build the expression `"2-3"`.

Return the number of different **expressions** that you can build, which evaluates to `target`.

Example 1:

Input: `nums = [1,1,1,1,1], target = 3`

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of `nums` be `target` 3.

- `-1 + 1 + 1 + 1 + 1 = 3`
- `+1 - 1 + 1 + 1 + 1 = 3`
- `+1 + 1 - 1 + 1 + 1 = 3`
- `+1 + 1 + 1 - 1 + 1 = 3`
- `+1 + 1 + 1 + 1 - 1 = 3`

Example 2:

Input: `nums = [1], target = 1`

Output: 1

Constraints:

- `1 <= nums.length <= 20`
- `0 <= nums[i] <= 1000`
- `0 <= sum(nums[i]) <= 1000`

Testcase 1: Test Result Runtime: 0 ms

Accepted Case 1 Case 2

Input:

`nums =`
`[1,1,1,1,1]`

target =
`3`

Output:
`5`

Expected:
`5`

Contribute a testcase

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm

```
#include <stdio.h>

#define INF 99999

void floydWarshall(int graph[][10], int n) {
    int dist[10][10];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    printf("Shortest distance matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
    }
```

```

        printf("\n");
    }
}

int main() {
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int graph[10][10];

    printf("Enter adjacency matrix (use %d for INF):\n", INF);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    floydWarshall(graph, n);

    return 0;
}

```

Output:

```

1 #include <stdio.h>
2
3 #define INF 99999
4
5
6 void floydWarshall(int graph[][10], int n) {
7     int dist[10][10];
8
9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            dist[i][j] = graph[i][j];
12        }
13    }
14
15    for (int k = 0; k < n; k++) {
16        for (int i = 0; i < n; i++) {
17            for (int j = 0; j < n; j++) {
18                if (dist[i][k] + dist[k][j] < dist[i][j]) {
19                    dist[i][j] = dist[i][k] + dist[k][j];
20                }
21            }
22        }
23    }
24
25    printf("Shortest distance matrix:\n");
26    for (int i = 0; i < n; i++) {
27        for (int j = 0; j < n; j++) {
28            if (dist[i][j] == INF)
29                printf("INF ");
30            else
31                printf("%d ", dist[i][j]);
32        }
33        printf("\n");
34    }
35
36 }
37
38
39 int main() {
40     int n;
41
42     printf("Enter number of vertices: ");
43     scanf("%d", &n);
44
45     int graph[10][10];
46
47     printf("Enter adjacency matrix (use %d for INF):\n", INF);
48     for (int i = 0; i < n; i++) {
49         for (int j = 0; j < n; j++) {
50             scanf("%d", &graph[i][j]);
51         }
52     }
53
54     floydWarshall(graph, n);
55
56     return 0;
57 }

```

```

Enter number of vertices: 4
Enter adjacency matrix (use 99999 for INF):
0 3 99999 5
2 0 99999 0
99999 6 0 99999
99999 99999 2 0
Shortest distance matrix:
0 3 7 5
2 0 6 4
8 6 0 10
10 0 2 0
Process returned 0 (0x0)   execution time : 60.326 s
Press any key to continue.

```

Leet Code exercise on Floyd's algorithm-1334

```
#define INF 1000000000

int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {

    int dist[n][n];

    // Initialize matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }

    // Fill edges
    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];

        dist[u][v] = w;
        dist[v][u] = w;
    }

    // Floyd-Warshall
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    int city = -1;
    int minReachable = INF;

    for (int i = 0; i < n; i++) {
        int count = 0;

        for (int j = 0; j < n; j++) {
```

```

        if (i != j && dist[i][j] <= distanceThreshold)
            count++;
    }

    if (count <= minReachable) {
        minReachable = count;
        city = i;
    }
}

return city;
}

```

Output:

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

Medium Topics Companies Hint

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

Input: `n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4`
Output: `3`
Explanation: The figure above describes the graph. The neighboring cities at a `distanceThreshold = 4` for each city are:
City 0 $\rightarrow$ [City 1, City 2]
City 1 $\rightarrow$ [City 0, City 2, City 3]
City 2 $\rightarrow$ [City 0, City 1, City 3]
City 3 $\rightarrow$ [City 1, City 2]
Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.

```

1  #define INF 1000000000
2
3  int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
4
5      int dist[n][n];
6
7      // Initialize matrix
8      for (int i = 0; i < n; i++) {
9          for (int j = 0; j < n; j++) {
10             if (i == j)

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`n =`
4

`edges =`
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

`distanceThreshold =`
4

Output

3

Expected

3

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

Medium Topics Companies Hint

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

Input: `n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4`
Output: `3`
Explanation: The figure above describes the graph. The neighboring cities at a `distanceThreshold = 4` for each city are:
City 0 $\rightarrow$ [City 1, City 2]
City 1 $\rightarrow$ [City 0, City 2, City 3]
City 2 $\rightarrow$ [City 0, City 1, City 3]
City 3 $\rightarrow$ [City 1, City 2]
Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.

```

1  #define INF 1000000000
2
3  int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
4
5      int dist[n][n];
6
7      // Initialize matrix
8      for (int i = 0; i < n; i++) {
9          for (int j = 0; j < n; j++) {
10             if (i == j)

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`n =`
5

`edges =`
[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]

`distanceThreshold =`
2

Output

0

Expected

0

Lab program 8:

A) . Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>

#define MAX 10
#define INF 9999

int main() {
    int n, cost[MAX][MAX], visited[MAX] = {0};
    int i, j, ne = 1;
    int min, mincost = 0;
    int a = 0, b = 0;

    printf("Enter number of vertices:");
    scanf("%d",&n);

    printf("Enter the cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    visited[0] = 1;
    printf("Edges in MST:\n");
    while(ne < n) {
        min = INF;
        for(i = 0; i < n; i++) {
            if(visited[i]) {
                for(j = 0; j < n; j++) {
```

```

if(!visited[j] && cost[i][j] < min) {
    min = cost[i][j];
    a = i;
    b = j;
}
}
}
}

printf("%d -> %d = %d\n" ,a, b, min);

visited[b] = 1;

mincost += min;

ne++;
}

printf("\nMinimum cost of spanning tree = %d\n", mincost);

return 0;
}

```

Output:

The screenshot shows a C++ IDE with a file named 'kruskals algo.c'. The code implements Kruskal's algorithm to find the Minimum Spanning Tree (MST) of a weighted undirected graph. It uses a priority queue (min-heap) to select edges in increasing order of weight, skipping those that would either create a cycle or result in a vertex with a degree greater than 2.

The output window shows the following execution results:

```

Enter number of vertices:4
Enter the cost adjacency matrix:
0 10 15 20
15 0 5 10
40 20 0 10
5 15 14 0
Edges in MST:
0 -> 1 = 10
1 -> 2 = 5
1 -> 3 = 10

Minimum cost of spanning tree = 25

Process returned 0 (0x0)   execution time : 50.422 s
Press any key to continue.

```

B) . Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>

int parent[10];

int find(int i) {
while(parent[i])
i = parent[i];
return i;
}

int uni(int i, int j) {
if(i != j) {
parent[j] = i;
return 1;
}
return 0;
}

int main() {
int n, cost[10][10];
int i, j, min, mincost = 0;
int a, b, u, v, ne = 0;
printf("Enter number of vertices:");
scanf("%d", &n);
printf("Enter cost matrix:\n");
for(i = 0; i < n; i++)
for(j = 0; j < n; j++) {
scanf("%d", &cost[i][j]);
if(cost[i][j] == 0)
cost[i][j] = 999;
}
printf("\nEdges selected:\n");
```

```

while(ne < n - 1) {
    min = 999;
    for(i = 0; i < n; i++)
    for(j = 0; j < n; j++)
    if(cost[i][j] < min) {
        min = cost[i][j];
        a = u = i;
        b = v = j;
    }
    u = find(u);
    v = find(v);
    if(uni(u, v)) {
        printf("%d -> %d = %d\n", a, b, min);
        mincost += min;
        ne++;
    }
    cost[a][b] = cost[b][a] = 999;
}
printf("\nMinimum cost = %d\n", mincost);
}

```

Output:

```

1  #include<stdio.h>
2  int parent[10];
3
4  int find(int i) {
5      while(parent[i])
6          i = parent[i];
7      return i;
8  }
9
10 int uni(int i, int j) {
11     if(i == j) {
12         parent[i] = i;
13         return 1;
14     }
15     return 0;
16 }
17 int main() {
18     int n, cost[10][10];
19     int i, j, min, mincost = 0;
20     int a, b, u, v, ne = 0;
21     printf("Enter number of vertices:");
22     scanf("%d", &n);
23     printf("Enter cost matrix:\n");
24     for(i = 0; i < n; i++) {
25         for(j = 0; j < n; j++) {
26             scanf("%d", &cost[i][j]);
27             if(cost[i][j] == 0)
28                 cost[i][j] = 999;
29         }
30     }
31     printf("\nEdges selected:\n");
32     while(ne < n - 1) {
33         min = 999;
34         for(i = 0; i < n; i++)
35         for(j = 0; j < n; j++)
36             if(cost[i][j] < min) {
37                 min = cost[i][j];
38                 a = u = i;
39                 b = v = j;
40             }

```

```

C:\Users\BMSCECSE-SH\Desktop
Enter number of vertices:4
Enter cost matrix:
0 10 15 20
15 0 5 10
40 20 0 10
5 15 14 0

Edges selected:
1 -> 2 = 5
3 -> 0 = 5
0 -> 1 = 10

Minimum cost = 20

Process returned 0 (0x0)   execution time : 29.039 s
Press any key to continue.

```

Lab program 9:

Implement fractional knapsack problem using Greedy technique.

```
#include <stdio.h>

struct Item {
    int value;
    int weight;
    float ratio;
};

void sortItems(struct Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (items[i].ratio < items[j].ratio) {
                struct Item temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

float fractionalKnapsack(struct Item items[], int n, int capacity) {
    float totalValue = 0.0;

    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {

            totalValue += items[i].value;
            capacity -= items[i].weight;
        } else {

            totalValue += items[i].ratio * capacity;
```

```

        break;
    }
}
return totalValue;
}
int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];

    printf("Enter value and weight of each item:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

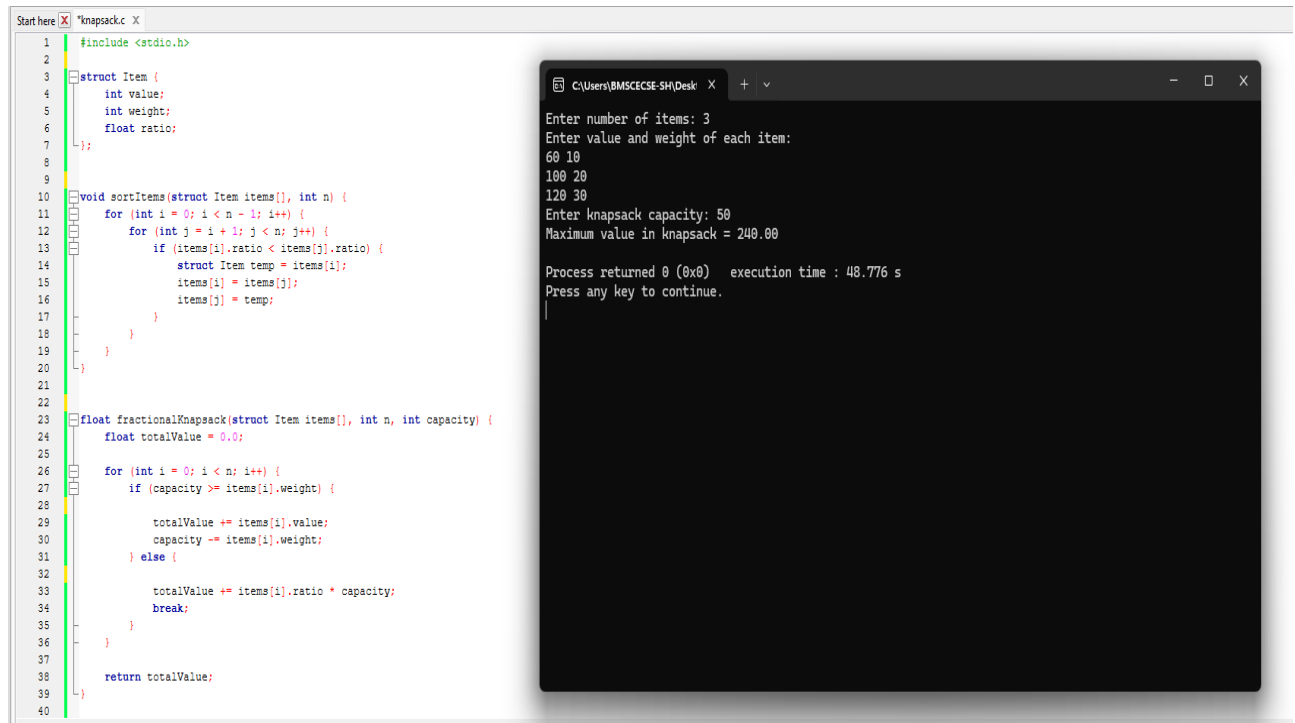
    sortItems(items, n);
    float maxVal = fractionalKnapsack(items, n, capacity);

    printf("Maximum value in knapsack = %.2f\n", maxVal);

    return 0;
}

```

Output:



```
1 #include <stdio.h>
2
3 struct Item {
4     int value;
5     int weight;
6     float ratio;
7 };
8
9
10 void sortItems(struct Item items[], int n) {
11     for (int i = 0; i < n - 1; i++) {
12         for (int j = i + 1; j < n; j++) {
13             if (items[i].ratio < items[j].ratio) {
14                 struct Item temp = items[i];
15                 items[i] = items[j];
16                 items[j] = temp;
17             }
18         }
19     }
20 }
21
22
23 float fractionalKnapsack(struct Item items[], int n, int capacity) {
24     float totalValue = 0.0;
25
26     for (int i = 0; i < n; i++) {
27         if (capacity >= items[i].weight) {
28             totalValue += items[i].value;
29             capacity -= items[i].weight;
30         } else {
31             totalValue += items[i].ratio * capacity;
32             break;
33         }
34     }
35
36     return totalValue;
37 }
38
39
40
```

```
C:\Users\BMSCECSE-SH\Desktop >
Enter number of items: 3
Enter value and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50
Maximum value in knapsack = 240.00

Process returned 0 (0x0)   execution time : 48.776 s
Press any key to continue.
```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n) {
    int min = INT_MAX, min_index;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] <= min) {
            min = dist[i];
            min_index = i;
        }
    }

    return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX];
    int visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
```



```

        for (int v = 0; v < n; v++) {
            if (!visited[v] &&
                graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
    int n, graph[MAX][MAX], src;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 if no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter source vertex: ");

```

```
scanf("%d", &src);
```

```
dijkstra(graph, n, src);
```

```
return 0;
```

```
}
```

Output:

The image shows a C program for Dijkstra's algorithm and its execution. The program is written in a text editor with line numbers 1 to 40. It includes headers for `<stdio.h>` and `<limits.h>`, and defines `MAX` as 100. The `minDistance` function finds the vertex with the minimum distance that has not been visited. The `dijkstra` function implements the algorithm by iteratively selecting the vertex with the minimum distance and updating the distances of its neighbors. The main function prompts the user for the number of vertices, the adjacency matrix, and the source vertex, then calls `dijkstra` and prints the shortest distances from the source to all other vertices.

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 #define MAX 100
5
6 int minDistance(int dist[], int visited[], int n) {
7     int min = INT_MAX, min_index;
8
9     for (int i = 0; i < n; i++) {
10         if (!visited[i] && dist[i] <= min) {
11             min = dist[i];
12             min_index = i;
13         }
14     }
15     return min_index;
16 }
17
18 void dijkstra(int graph[MAX][MAX], int n, int src) {
19     int dist[MAX];
20     int visited[MAX];
21
22     for (int i = 0; i < n; i++) {
23         dist[i] = INT_MAX;
24         visited[i] = 0;
25     }
26
27     dist[src] = 0;
28
29     for (int count = 0; count < n - 1; count++) {
30         int u = minDistance(dist, visited, n);
31         visited[u] = 1;
32
33         for (int v = 0; v < n; v++) {
34             if (!visited[v] &&
35                 graph[u][v] &&
36                 dist[u] != INT_MAX &&
37                 dist[u] + graph[u][v] < dist[v]) {
38
39                 dist[v] = dist[u] + graph[u][v];
40             }
41         }
42     }
43 }
```

The execution output shows the program running in a terminal window. It prompts for the number of vertices (4), the adjacency matrix (0 10 0 5, 0 0 1 2, 0 0 0 4, 0 3 9 0), and the source vertex (0). It then displays the shortest distances from the source to all other vertices: 0 0, 1 8, 2 9, 3 5. The process returned 0 (0x0) and the execution time was 141.633 s. The user is prompted to press any key to continue.

```
Enter number of vertices: 4
Enter adjacency matrix (0 if no edge):
0 10 0 5
0 0 1 2
0 0 0 4
0 3 9 0
Enter source vertex: 0
Vertex Distance from Source
0 0
1 8
2 9
3 5

Process returned 0 (0x0)   execution time : 141.633 s
Press any key to continue.
```

Lab program 11:**Implement “N-Queens Problem” using Backtracking.**

```
#include <stdio.h>

#define MAX 20

int board[MAX][MAX];
int N;

int isSafe(int row, int col) {
    int i, j;
    for (i = 0; i < row; i++) {
        if (board[i][col])
            return 0;
    }
    for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j])
            return 0;
    }
    for (i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++) {
        if (board[i][j])
            return 0;
    }
    return 1;
}

int solveNQueens(int row) {
    int col;
    if (row == N)
        return 1;
    for (col = 0; col < N; col++) {
```

```

        if (isSafe(row, col)) {
            board[row][col] = 1;
            if (solveNQueens(row + 1))
                return 1;
            board[row][col] = 0;
        }
    }

    return 0;
}

void printBoard() {
    int i, j;
    printf("\nSolution:\n\n");

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (board[i][j])
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}

int main() {

    printf("Enter value of N: ");
    scanf("%d", &N);

```

```

    if (solveNQueens(0))

        printBoard();

    else

        printf("No solution exists.\n");


    return 0;

}

```

Output:

The screenshot shows a C++ IDE with a file named 'nqueens.c' and an output window. The code in 'nqueens.c' is as follows:

```

1  #include <stdio.h>
2
3  #define MAX 20
4
5  int board[MAX][MAX];
6  int N;
7
8
9  int isSafe(int row, int col) {
10
11     int i, j;
12
13     for (i = 0; i < row; i++) {
14         if (board[i][col])
15             return 0;
16     }
17
18     for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
19         if (board[i][j])
20             return 0;
21     }
22
23     for (i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++) {
24         if (board[i][j])
25             return 0;
26     }
27
28     return 1;
29 }
30
31 int solveNQueens(int row) {
32
33     int col;
34
35     if (row == N)
36         return 1;
37
38     for (col = 0; col < N; col++) {
39         if (isSafe(row, col)) {
40             board[row][col] = 1;
41             if (solveNQueens(row + 1))
42                 return 1;
43             board[row][col] = 0;
44         }
45     }
46     return 0;
47 }
48
49 int main() {
50     printf("Enter value of N: ");
51     scanf("%d", &N);
52     printf("Solution:\n");
53     solveNQueens(0);
54     printf("Process returned 0 (0x0)   execution time : 1.512 s\n");
55     printf("Press any key to continue.\n");
56     getch();
57 }

```

The output window shows the following text:

```

Enter value of N: 5
Solution:
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

Process returned 0 (0x0)   execution time : 1.512 s
Press any key to continue.

```